Testing database_implementation.sql locally

CROSSFADE, info579_group_1 (Herling, Nathan; Pabari, Nidhi Nilesh). INFO 579 Project 2, Task 1. 228 tests; last run 2026-11-08 20:01:18: 228 passed, 0 failed, on MySQL 8.0.46-0ubuntu0.24.04.4.

How database_implementation.sql is checked: the script is run twice, then the database it leaves behind is compared with the Project 1 physical model, the requirements analysis and the sample data, and every key and rule is made to refuse a bad write.

1. What the tests compare against

Source	From	How it is used
Physical model	physical_data_model.png (Project 1)	Typed by hand into SPEC in test_implementation.py: tables, columns in order, types and sizes, NULL / NOT NULL, keys, the 13 relationship lines, the uq_ indexes. ENUM values and UNSIGNED flags come from the data dictionary, because the diagram abbreviates them. Never read from the .sql file.
Value rules	requirements_analysis.pdf (Project 1)	Each CHECK is listed with a value it must refuse and where the rule comes from. Where the analysis is silent (payout rate positive, email has an @), the rule is the one the script's own comment states, and is marked as such.
Sample data	data.xlsx → data.csv (Project 1)	The graded workbook, exported cell for cell and checked against itself by tools/export_p1_data.py. Rows are matched by primary key; numbers compared as exact decimals.

2. Run them

a. What you need. A running MySQL Server 8.0.16 or later (CHECK constraints) and its root password; Python 3.10 or later. The mysql command-line client and Workbench are not needed.

b. Get the files. Download crossfade_impl_tests.zip with the button above and extract it. It holds one folder, crossfade_impl_tests, with the tests, database_implementation.sql, data.csv and this guide. (Or take the files one by one; they only need to be in the same folder.)

```
Expand-Archive -Force $HOME\Downloads\crossfade_impl_tests.zip .
cd .\crossfade_impl_tests
```

c. Set up Python (once). A virtual environment with the three packages. macOS / Linux: python3 and .venv/bin/python.

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
```

d. Run the tests. Give the tests your MySQL password, then run. These tests drop and rebuild the crossfade database from database_implementation.sql (twice), and leave it loaded; nothing else on the server is touched.

```
$env:CROSSFADE_DB_PASSWORD = "your-root-password"
.venv\Scripts\python -m pytest
```

e. Read the result. The header names the two files it found; a counter shows each statement of the script as it runs; then one line per test and a summary. Every failure says which column, key, rule or cell differs, and what MySQL said.

```
crossfade impl  ...\database_implementation.sql
      running database_implementation.sql: statement 22/22
collected 228 items
test_implementation.py::test_load_first_run PASSED   [  0%]
...
========== 228 passed in 0.8s ==========
```

f. Without Python. From a clone of the repository, the mysql client alone: runs the script twice, compares every row with data.xlsx and checks the integrity rules; ends with PASS.

```
python project2/scripts/check_implementation.py --mysql "C:/Program Files/MySQL/MySQL Server 8.4/bin/mysql.exe"
```

3. What each group proves

Group	Tests	What it proves
load	6/6	The script runs start to finish, and again on top of itself (re-runnable). No warnings: MySQL warns when it silently rounds or trims a value, so a clean load means every value went in as written. A purpose comment before every statement (the brief). Checks are never switched off. Each table created once and loaded after its parents.
schema	41/41	Exactly the ten tables. Every column in the diagram's order with its exact type, size, UNSIGNED flag and NULL / NOT NULL. Primary keys, composite on the four associative tables. All 13 foreign keys: columns, parent, name and ON DELETE rule. The five unique keys. Constraint names follow fk_ / uq_ / ck_.
read	10/10	One live SELECT per table: the columns come back in order and every value has the right Python type (Decimal, never float; dates as dates; the checksum as 32 bytes; BIT as one byte).
data	42/42	Every table has the row count and every cell of data.csv (the Project 1 data.xlsx). The nine cross-row integrity checks from the requirements analysis hold. No child row points at a missing parent.
enforce	129/129	Inside a transaction that is always rolled back: a valid new row is accepted, then one bad write per rule is refused with the exact MySQL error and constraint name. Duplicate keys (1062), orphans (1452), NULLs (1048), CHECKs (3819), type limits (1406, 1264, 1265, 1292), and the ON DELETE behaviours (CASCADE, SET NULL, refused with 1451).

4. Files

File	Role
database_implementation.sql	the file under test (Task 1)
test_implementation.py	the tests: the spec typed from the physical model, the value rules, all five groups
conftest.py	shared set-up: finds the files, connects to MySQL, splits the script into statements, writes the results snapshot
data.csv	the Project 1 sample data, exported cell for cell from the graded data.xlsx: the expected rows
pytest.ini	pytest settings (verbose, no cache folder)
requirements.txt	pytest, PyMySQL, cryptography (MySQL 8 sign-in)
test_implementation_guide.pdf	how to run the tests locally, what they cover and what they could miss
check_implementation.py	the no-Python-driver check: the mysql client only (runs the script twice, compares every row, checks the integrity rules)
mutate_implementation.py	tests the tests: breaks the script 19 ways, one at a time, and checks every break is caught (repo only)
crossfade_impl_tests.zip	the tests, both input files and the guide, in one folder that runs as-is

5. What the tests could miss

Passing means the file agrees with its sources and refuses what they forbid. It does not mean the following:

Gap	What could slip through	What covers it instead
A mistake in the physical model itself	The schema tests prove the database matches physical_data_model.png, not that the model is the best design. If the diagram has a poor type choice, the script and the tests agree with it.	The model was graded in Project 1; type choices are argued column by column in the data dictionary.
A mistake shared by the requirements analysis and the script	ENUM values and UNSIGNED flags are taken from the data dictionary (built from the analysis), because the diagram abbreviates them. If the analysis is wrong there, so is the spec.	check_consistency.py cross-checks the analysis, data.xlsx and the dictionary column by column.
Wrong sample data that was handed in wrong	The data tests prove the database holds exactly data.xlsx. A value that was wrong in the graded workbook passes. Known case: scoring_model 2 (beardown_rrm_cfg12) is recorded as concat; its model files say gated.	Listed on the History page under Known discrepancies; the brief requires the data as provided.

Gap	What could slip through	What covers it instead
Running in Workbench is not the same run	The tests run the script statement by statement through PyMySQL, split by our own splitter. Workbench has its own splitter and session settings (for example Safe Updates). A statement that splits differently there would not be caught.	check_implementation.py runs it through the mysql client; run it once in Workbench by hand before submitting.
Different server settings	Strict SQL mode is assumed (the MySQL 8 default). On a server without it, a bad value is stored with a warning instead of refused, so the type-limit tests would fail there, correctly but confusingly.	Each run records the server version and sql_mode; the site page shows the last run's.
Accepted boundary values	Each CHECK is tested with one value it must refuse, not with the edge values it must accept. A rule written as < 1 where the analysis says at most 1 would still refuse 1.2 and pass, while wrongly refusing exactly 1.00.	The sample data contains some edge values (a blend weight of 1.00, a royalty of 0), which the load accepts.
Rules across rows are not enforced by the schema	I1, I2, I3, I5 and I7 are checked on the loaded data only. Nothing in the tables stops a later insert from breaking them: by design, the analysis gives them to stored procedures.	Task 4 (routines.sql) and its tests.
ON DELETE rules are the script's choice	The analysis does not say what happens on delete. The tests check the three rules the script's comments state (two CASCADE, one SET NULL), so they cannot judge whether those are the right business choices.	Argued in the script's table comments.
Things not looked at	Collation and case (whether two stage names differing only in case count as duplicates), ON UPDATE rules, secondary indexes beyond the keys, users and privileges.	Not required by the brief.

6. Testing the tests

mutate_implementation.py copies the file, makes one deliberate mistake, and runs every test against the copy. A mistake is caught when a test fails that passes on the real file. 19 of 19 were caught.

Mistake	Caught	First failing tests
a column one character wider than the model (genre.name VARCHAR(10))	yes (1 failed)	test_schema_columns[genre]
UNSIGNED missing on a key (track.track_id INT)	yes (200 failed)	test_load_first_run, test_load_second_run, test_schema_tables
a CHECK left out (I4, the ambiguity window)	yes (2 failed)	test_schema_constraint_names, test_enforce_check[ck_station_ambiguity_window]
an extra column in a composite primary key (play)	yes (1 failed)	test_schema_primary_key[play]
an optional column made NOT NULL (play.reaction)	yes (161 failed)	test_load_first_run, test_load_second_run, test_schema_columns[play]
a required column left nullable (track.title)	yes (2 failed)	test_schema_columns[track], test_enforce_not_null[track-title]
ON DELETE CASCADE left out (station_blend -> station)	yes (2 failed)	test_schema_foreign_key[fk_station_blend_station], test_enforce_delete_station_cascades_blend_and_keeps_plays
a foreign key under another name (fk_play_stn)	yes (3 failed)	test_schema_foreign_key[fk_play_station], test_schema_no_extra_foreign_keys, test_enforce_orphan_refused[fk_play_station]
a foreign key left out (track -> artist)	yes (4 failed)	test_schema_foreign_key[fk_track_artist], test_schema_no_extra_foreign_keys, test_enforce_orphan_refused[fk_track_artist]
a unique key left out (listener.email)	yes (2 failed)	test_schema_unique_key[uq_listener_email], test_enforce_duplicate_unique[uq_listener_email]
foreign-key checks switched off during the load	yes (3 failed)	test_load_every_statement_commented, test_load_never_switches_checks_off, test_load_inventory
DROP DATABASE IF EXISTS removed (the script cannot run twice)	yes (4 failed)	test_load_first_run, test_load_second_run, test_load_no_warnings

Mistake	Caught	First failing tests
a DECIMAL one place too short, so MySQL rounds the data (probability DECIMAL(5,3))	yes (5 failed)	test_load_no_warnings, test_schema_columns[genre_probability], test_data_every_cell[genre_probability]
an ENUM value the model does not have (plan_tier 'family')	yes (1 failed)	test_schema_columns[listener]
DATE stored as DATETIME (listener.signup_date)	yes (2 failed)	test_schema_columns[listener], test_data_every_cell[listener]
the fixed 32-byte checksum stored as VARBINARY(64)	yes (2 failed)	test_schema_columns[track], test_enforce_type_limit[track-audio_checksum-too-long]
one royalty amount changed (0.0035 -> 0.0036)	yes (2 failed)	test_data_every_cell[play], test_data_integrity_rule[I7-royalty-is-payout-rate-from-30-seconds]
one artist row left out (1049)	yes (3 failed)	test_data_row_count[artist], test_data_every_cell[artist], test_enforce_rollback_leaves_data
the purpose comment before one CREATE TABLE removed (the brief)	yes (1 failed)	test_load_every_statement_commented

7. Findings

Primary key names. The script writes CONSTRAINT pk_<table> PRIMARY KEY (...) for all ten tables, but MySQL ignores a name on a primary key and always calls it PRIMARY. The tests accept PRIMARY. The header comment ("Every constraint is named: pk_<table>, ...") promises names the database never has; fk_, uq_ and ck_ names do survive.